

VORTEX TECH

Cyber Security Internship — Week 3

Security Audit Report

OWASP Juice Shop Vulnerability Assessment

Prepared by

Hadi Faheem

September 16, 2026

Executive Summary

An automated security audit was conducted on the OWASP Juice Shop deliberately-vulnerable web application using OWASP ZAP v2.17.0. The scanning tool identified 9 vulnerabilities spanning multiple risk levels, with 1 High, 4 Medium, 3 Low, and 1 Informational finding.

This report documents the 3 most critical findings identified during the audit, including their real-world impact, affected components, and practical remediation strategies with code examples.

Scan Details

Target: http://localhost:3000 (OWASP Juice Shop)

Scanner: OWASP ZAP v2.17.0 by Checkmarx

Scan Date: Wednesday, 16 September 2026 02:19:42

Total Vulnerabilities Found: 9

Scope: Full automated active and passive scanning

Finding 1: SQL Injection (High Risk)

Summary

The application is vulnerable to SQL injection attacks. An attacker can manipulate SQL queries to bypass authentication, extract sensitive data, modify database contents, or execute arbitrary code on the backend server.

Vulnerability Details

CWE-89: Improper Neutralization of Special Elements Used in an SQL Command

OWASP Category: A03:2021 — Injection

Severity: High | Confidence: Low

Affected Endpoint: GET /rest/products/search?q=

How It Was Found

OWASP ZAP's active scanner tested the search parameter with SQL metacharacters:

```
Payload: '(  
Response: HTTP 500 Internal Server Error  
Evidence: Server error behavior indicates SQL injection possible
```

Real-World Impact

Authentication Bypass: Attacker logs in without credentials using 'OR 1=1' style payloads.

Data Theft: Entire user database, payment information, and personal data can be extracted.

Data Modification: Attackers can alter prices, add administrative accounts, or change transaction records.

Remote Code Execution: Some databases permit executing OS commands, enabling full server compromise.

Historical Reference: The Target breach (2013) involved SQL injection and resulted in the theft of 40 million credit card records.

Remediation Code

```
// VULNERABLE - DO NOT USE
const query = `SELECT * FROM users WHERE email = '${userInput}'`;
db.query(query, callback);

// SAFE - Use Prepared Statements
const query = 'SELECT * FROM users WHERE email = ?';
db.query(query, [userInput], callback);
```

Additional controls: Input validation, principle of least privilege for database users, WAF rules, regular SAST/DAST testing.

Finding 2: CSP Header Not Set (Medium Risk)

Summary

The application does not implement a Content Security Policy (CSP) HTTP header, leaving it vulnerable to Cross-Site Scripting (XSS) attacks and content injection vulnerabilities.

Vulnerability Details

CWE-693: Protection Mechanism Failure

OWASP Category: A05:2021 — Security Misconfiguration

Severity: Medium | Confidence: High

Finding: Missing Content-Security-Policy header in all responses

How It Was Found

OWASP ZAP's passive scanner analyzed HTTP response headers from multiple endpoints:

```
GET http://localhost:3000/
GET http://localhost:3000/sitemap.xml
Result: No Content-Security-Policy header present
```

Real-World Impact

XSS Exploitation: Malicious JavaScript injected via XSS can execute without restriction.

Session Hijacking: Attacker can steal cookies and impersonate users.

Credential Theft: Fake login forms can be injected to capture passwords.

Phishing: Application content can be modified to redirect users to malicious sites.

CSP acts as a safety net even if XSS vulnerabilities exist in the code.

Remediation Code

```
// Express.js with helmet library
const helmet = require('helmet');
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'"],
    styleSrc: ["'self'", "'unsafe-inline'"],
    imgSrc: ["'self'", "https:"]
  }
}));
```

Finding 3: Private IP Disclosure (Low Risk)

Summary

Internal private IP addresses are exposed in HTTP response bodies. This information aids attackers in network reconnaissance and targeting of internal systems.

Vulnerability Details

CWE-497: Exposure of Sensitive System Information to an Unauthorized Control Sphere

OWASP Category: A01:2021 — Broken Access Control

Severity: Low | Confidence: Medium

Evidence: Endpoint `/rest/admin/application-configuration` exposes `192.168.99.100:3000` and `192.168.99.100:4200`

How It Was Found

OWASP ZAP's passive scanner searched responses using RFC 1918 private IP address patterns:

```
Pattern: /\b(?:10|172(?:?:1[6-9]|2[0-9]|3[01])|192\.168)\.S+/g
Found: 192.168.99.100:3000
Location: API response body
```

Real-World Impact

Network Reconnaissance: Attackers learn internal infrastructure and service locations.

Targeted Attacks: When combined with SSRF vulnerabilities, specific internal services can be targeted.

Compliance Violation: PCI-DSS and HIPAA standards forbid exposing infrastructure details.

Attack Surface Mapping: Part of initial reconnaissance phase before exploitation.

Remediation Code

```
// VULNERABLE
app.get('/api/config', (req, res) => {
  res.json({ database: '192.168.1.5' });
});

// SAFE
app.get('/api/config', (req, res) => {
  res.json({ api_version: '1.0', features: ['auth'] });
  // Sensitive details logged server-side only
});
```

All Findings Summary

9 total vulnerabilities were identified:

High Risk (1): SQL Injection Medium Risk (4): CSP Header, CORS Misconfiguration, Anti-clickjacking Header, Session ID in URL Low Risk (3): Private IP Disclosure, Timestamp Disclosure, MIME Sniffing Informational (1): Modern Web Application

Recommendations

Immediate Actions: Implement parameterized queries to fix SQL injection vulnerability.

Short-Term: Deploy security headers (CSP, X-Frame-Options, X-Content-Type-Options).

Short-Term: Audit error messages and API responses for information leaks.

Ongoing: Regular SAST/DAST testing and security training for development team.

Compliance Note

This audit was conducted exclusively on OWASP Juice Shop, a deliberately vulnerable practice application designed for security training. No real-world production systems were tested without authorization. All findings and techniques documented are for educational purposes only.

About This Report

This professional security report was generated with the assistance of an AI system trained on OWASP security standards and best practices. All findings, vulnerabilities, remediation code examples, and recommendations are based on automated scanning results and established industry security guidance. Human review is recommended before implementing any remediation steps in production environments.

Author

Hadi Faheem